Lab 5: Static Code Analysis

Pylint, Bandit and Flake8 on the IT Helpdesk Ticket Allocator

Name	M Devashruth
SRN / Section	PES1UG24CS251 / 5E
Repository	SETAPESU26/L5_06_it_helpdesk
File analysed	helpdesk_system.py

Results at a glance

Tool	Before	After
Pylint	5.53/10 (21 messages)	10.00/10 (0 messages)
Bandit	4 issues (1 High, 3 Low)	No issues identified
Flake8	15 issues	0 issues (empty report)

Commits

Commit	Message
53018ea	Fix Bandit issues: remove shell=True, hardcoded password, try-except-pass
209bb5a	Fix Pylint issues: docstrings, naming, mutable default, with-open, validation
af1b1dd	Fix Flake8 issues: line length and final newline

Contents: 1 Setup and run, 2 Before reports, 3 Issue table, 4 Fixes (3 commits), 5 After reports, 6 Reflection, Appendix: final code.

1. Setup and run

My first attempt used the Linux commands from the handout (**python3**, **source**), which do not work in Windows PowerShell. I fixed it with **python -m venv venv**, allowing scripts for the session, and activating with **venv\Scripts\Activate.ps1**.

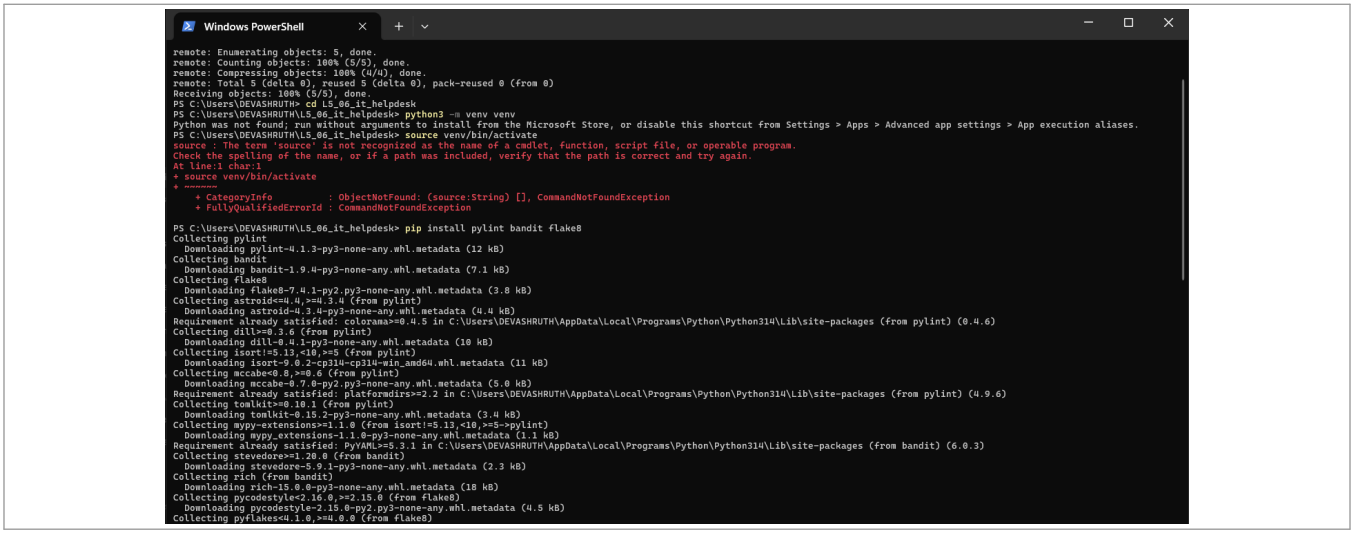


Fig 1: First attempt: python3 and source fail on Windows, tools installed globally.

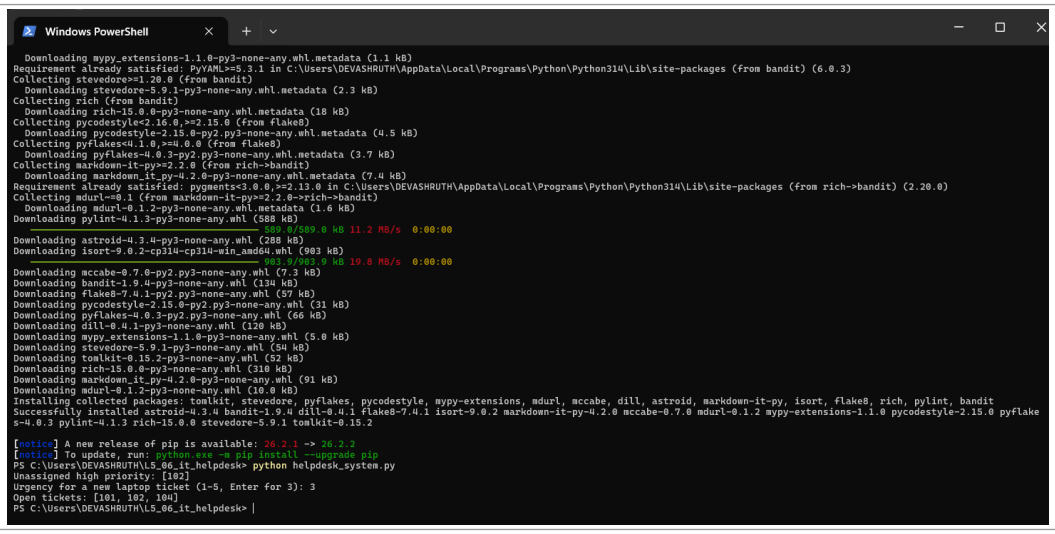


Fig 2: Script runs correctly (output before any changes).

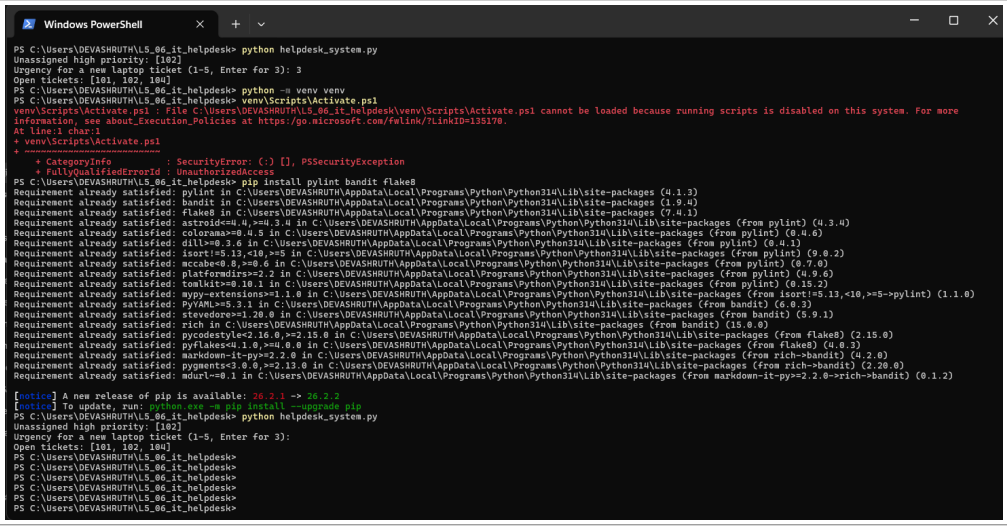


Fig 3: venv created, but PowerShell blocked Activate.ps1 (execution policy).

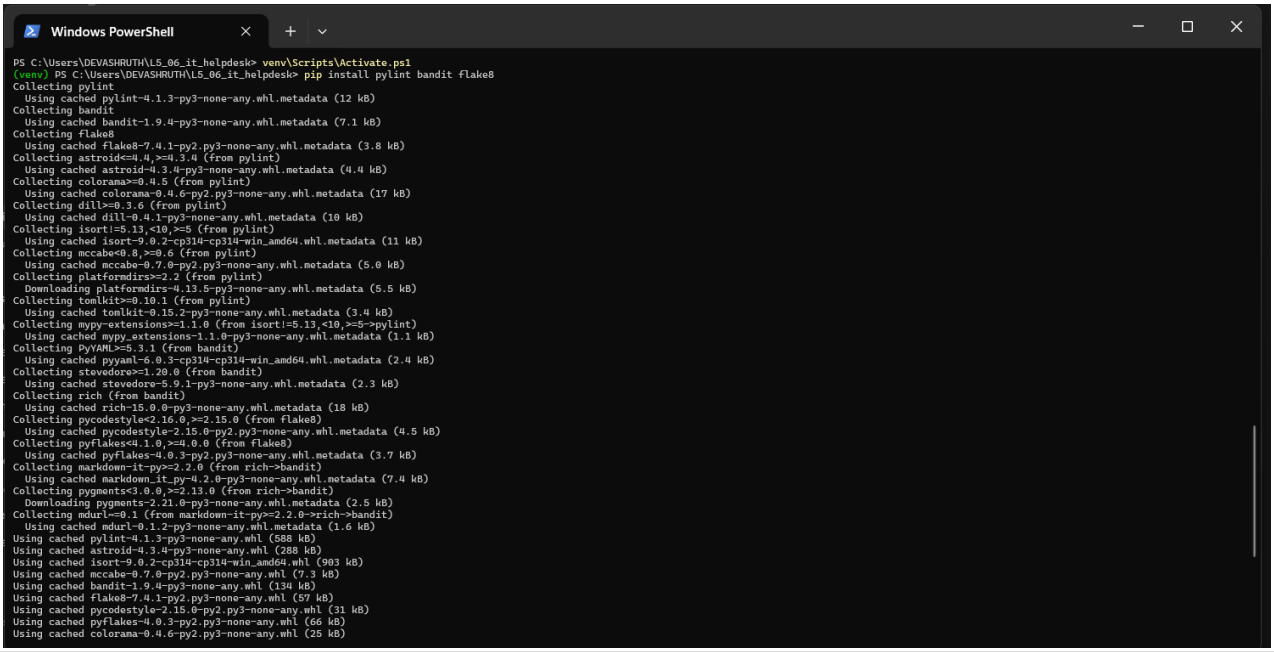


Fig 4: With the policy relaxed for the session, (venv) is active and pylint, bandit, flake8 install.

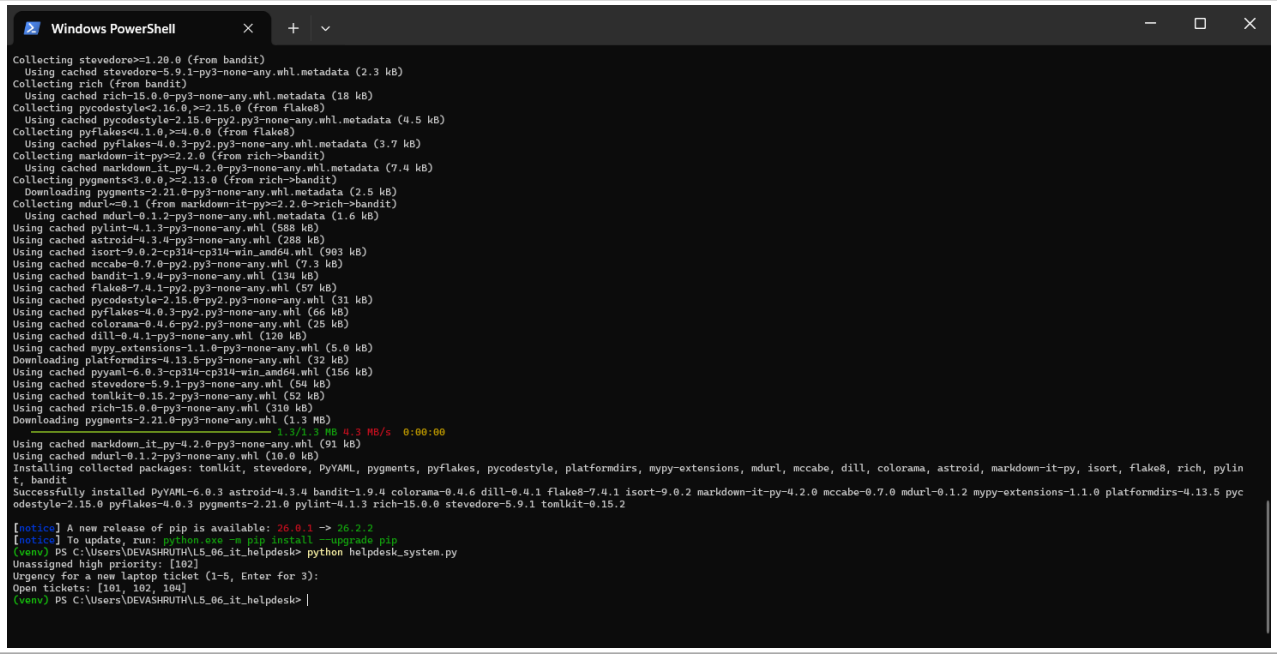


Fig 5: Installs finished inside (venv) and the script runs.

2. Static analysis reports: before

Run with output saved to the **before** folder (before\pylint_report.txt, bandit_report.txt, flake8_report.txt).

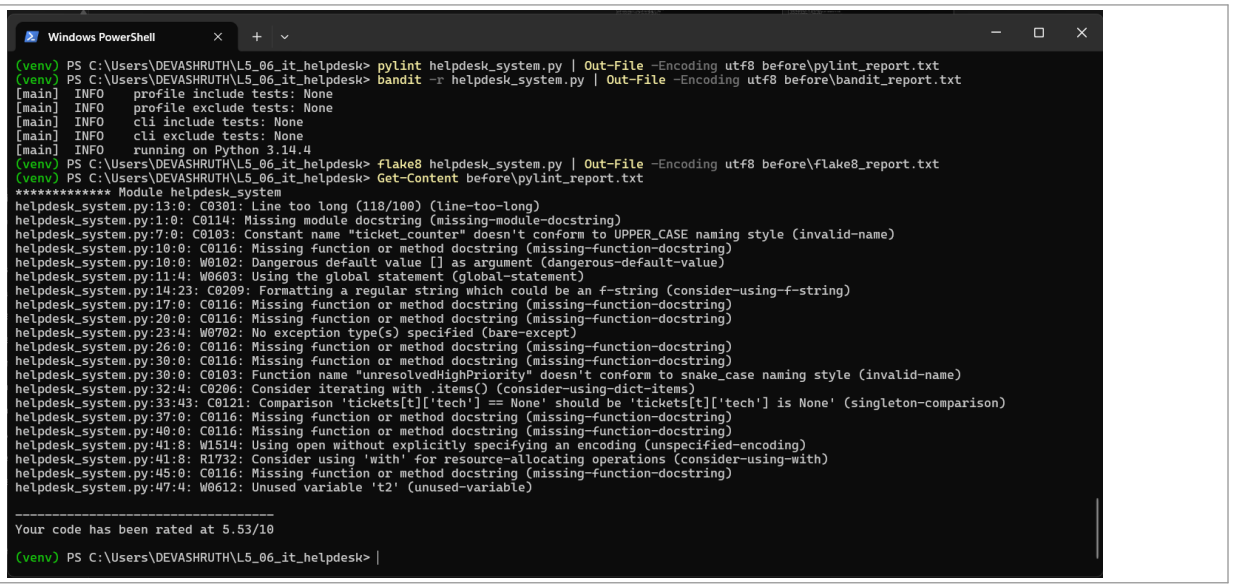


Fig 6: Pylint before: 5.53/10.

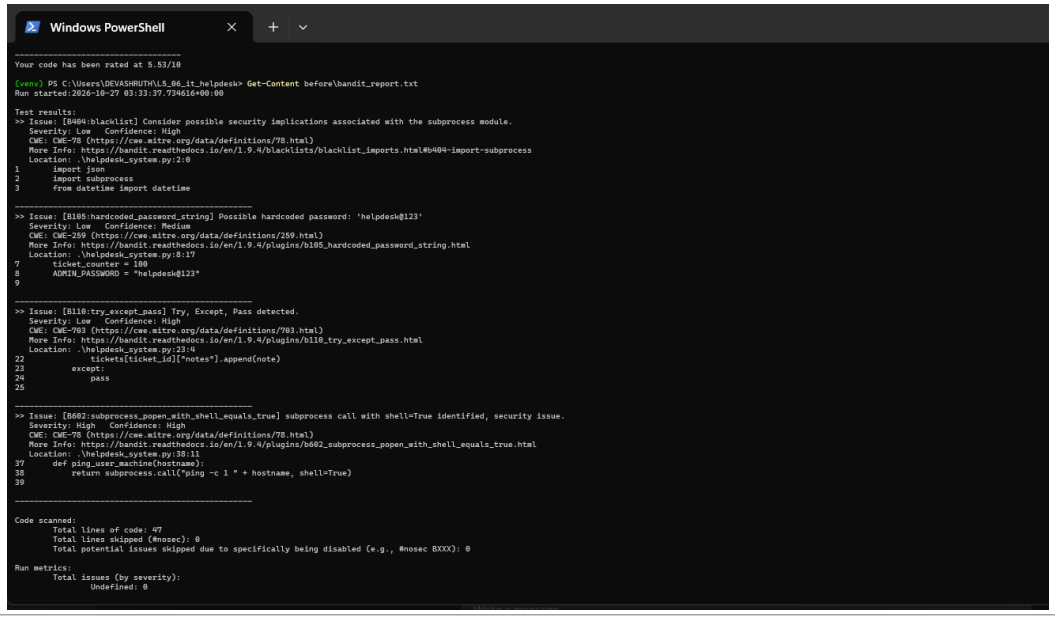


Fig 7: Bandit before: B404, B105, B110 (Low) and B602 (High, shell=True).

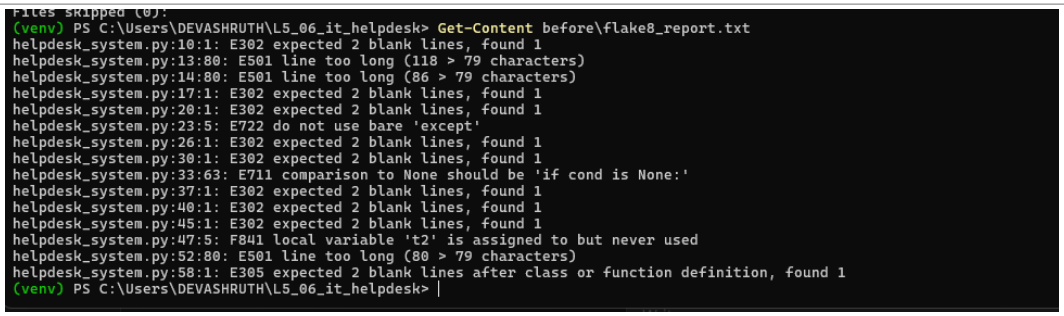


Fig 8: Flake8 before: 15 issues (E302, E501, E722, E711, F841, E305).

3. Issues identified and documented

Issue	Type	Line(s)	Description	Fix approach
Shell injection (B602, High)	Security	38	subprocess.call with shell=True and a joined string lets input run any command	Pass a list of arguments, drop shell=True, validate hostname
Hardcoded password (B105)	Security	8	Admin password stored in source code	Read it from the HELPDESK_ADMIN_PASSWORD environment variable
Try / except / pass (B110, W0702, E722)	Bug	23-24	Bare except hides every error, so add_note(999) fails silently	Catch KeyError only and print a clear message
subprocess import (B404, Low)	Security	2	Import of subprocess flagged	Acceptable once shell=True is gone; marked # nosec B404
Mutable default argument (W0102)	Bug	10	audit_trail=[] is shared across all calls	Default to None, create a new list inside the function
Resource not closed (R1732, W1514)	Bug	41-43	open() without with and without encoding	with open(file, 'w', encoding='utf-8')
Global statement, constant name (W0603, C0103)	Style	7, 11	Global counter, lowercase constant name	Use itertools.count (TICKET_IDS), no global needed
Comparison to None (C0121, E711)	Style	33	tech == None	Use is None
Function name (C0103)	Style	30	unresolvedHighPriority is camelCase	Rename to unresolved_high_priority
Missing docstrings (C0114, C0116)	Style	1, 10, 17, 20, 26, 30, 37, 40, 45	No module or function docstrings	Add short docstrings
Dict iteration (C0206)	Style	32	Loops over keys then indexes the dict	Use .items()
String formatting (C0209)	Style	14	Old % formatting	Use an f-string
Unused variable (W0612, F841)	Style	47	t2 assigned but never used	Call the function without storing the result
Formatting (E302, E305, E501, C0301)	Style	13, 14, 52 and many	Missing blank lines, lines over 79 characters	Add blank lines, wrap long lines, final newline

Bugs found by running the program (not reported by the tools)

Problem found by running the program	Why no tool flagged it	Fix
Priority -7 accepted for a ticket	Valid integer, so it is not a syntax or style problem	raise_ticket rejects urgency outside 1-5
close_ticket silently does nothing on a wrong password	Logic problem	Prints a message when the password is wrong or not set
ping -c 1 only works on Linux/Mac	Platform behaviour	Use -n on Windows, -c otherwise
int(urgency) crashes on non-numeric input	Runtime error	ValueError is caught in main and reported
main() runs when the file is imported	Design problem	Added if __name__ == '__main__' guard

4. Fixes (three commits)

Commit 1: Bandit (53018ea)

Removed shell=True (argument list plus hostname validation, -n/-c by platform), moved the password to an environment variable, replaced the bare except with except KeyError. Bandit now reports no issues and the script still runs.

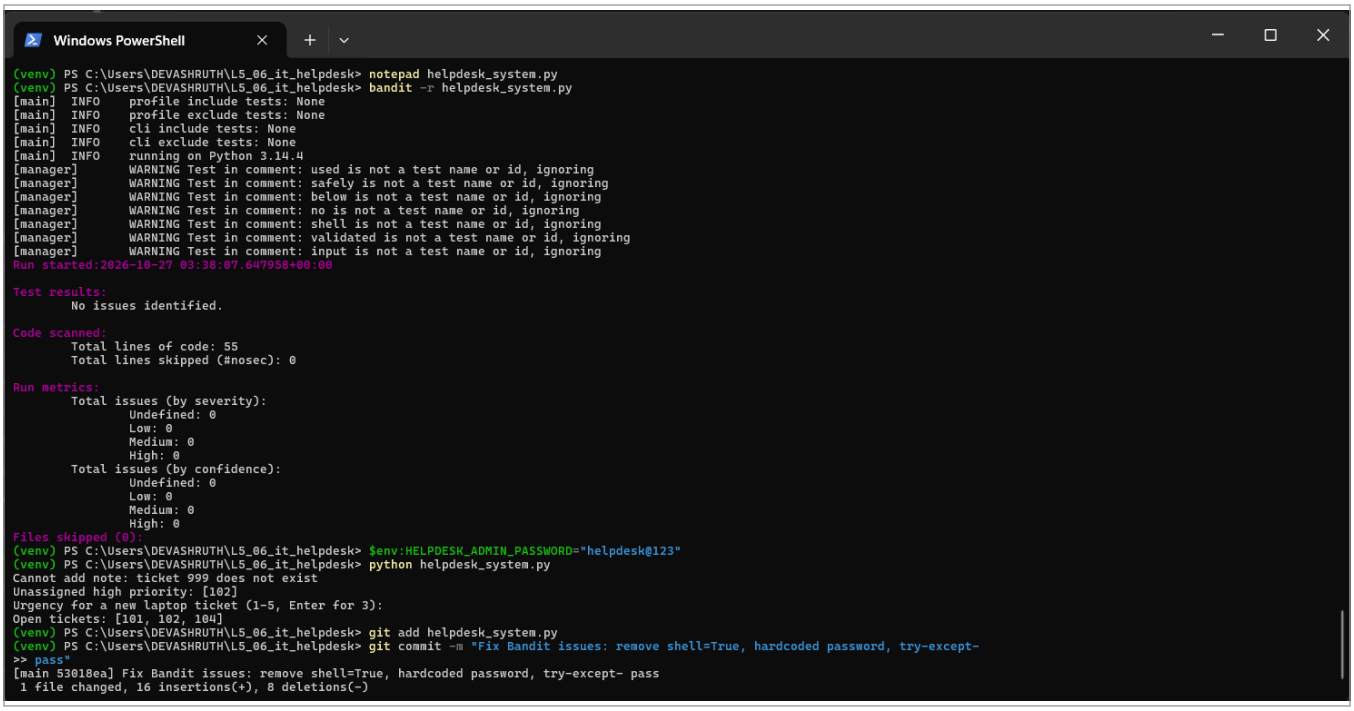


Fig 9: Bandit clean, script output, and commit 53018ea.

Commit 2: Pylint (209bb5a)

Fixed the mutable default, global statement, naming, docstrings, is None, f-strings, .items(), with open and encoding, unused variable, __main__ guard, and added urgency validation. Score 9.85/10 (only a missing final newline left), Bandit still clean.

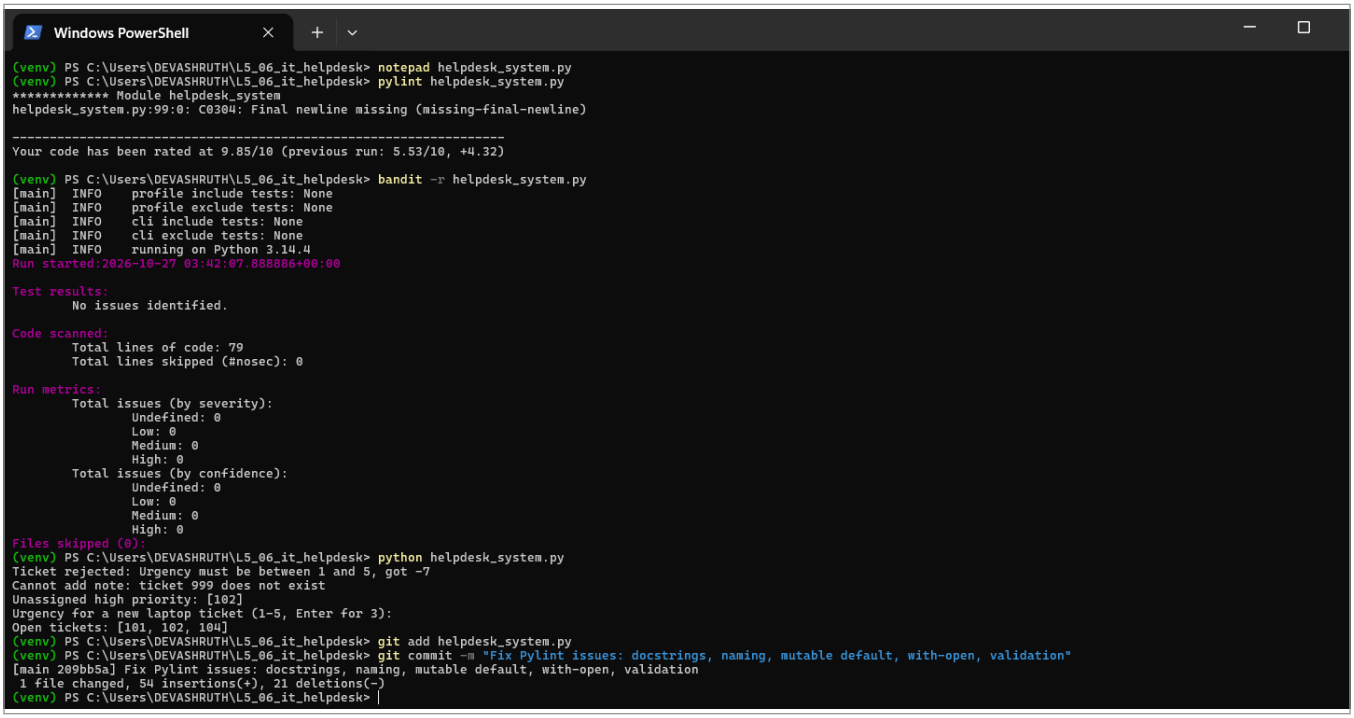


Fig 10: Pylint 9.85/10, Bandit clean, script running, and commit 209bb5a.

Commit 3: Flake8 (af1b1dd)

Wrapped the four lines over 79 characters and added the final newline. Flake8 prints nothing, Pylint is 10.00/10, Bandit is clean.

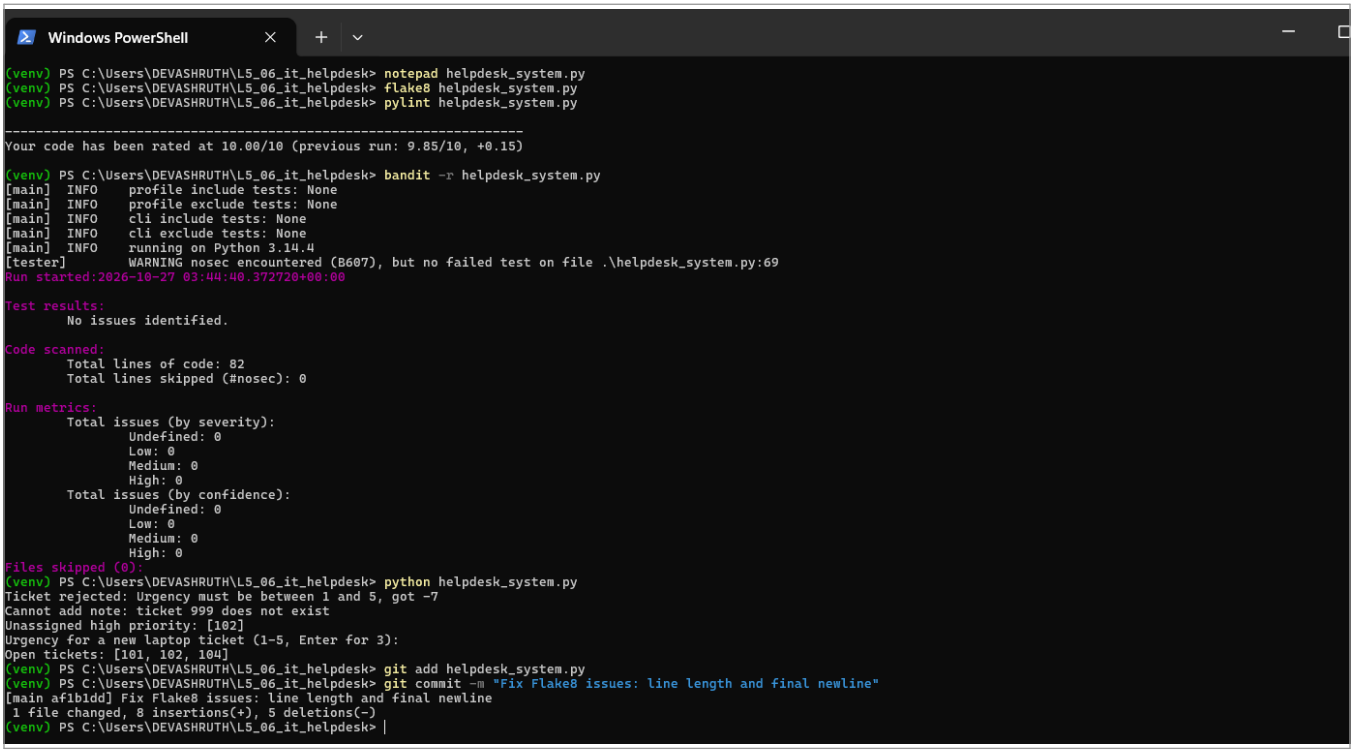


Fig 11: Flake8 empty, Pylint 10.00/10, Bandit clean, script running, and commit af1b1dd.

5. Static analysis reports: after

Saved to the **after** folder. The Flake8 report is 0 bytes because there is nothing left to report.

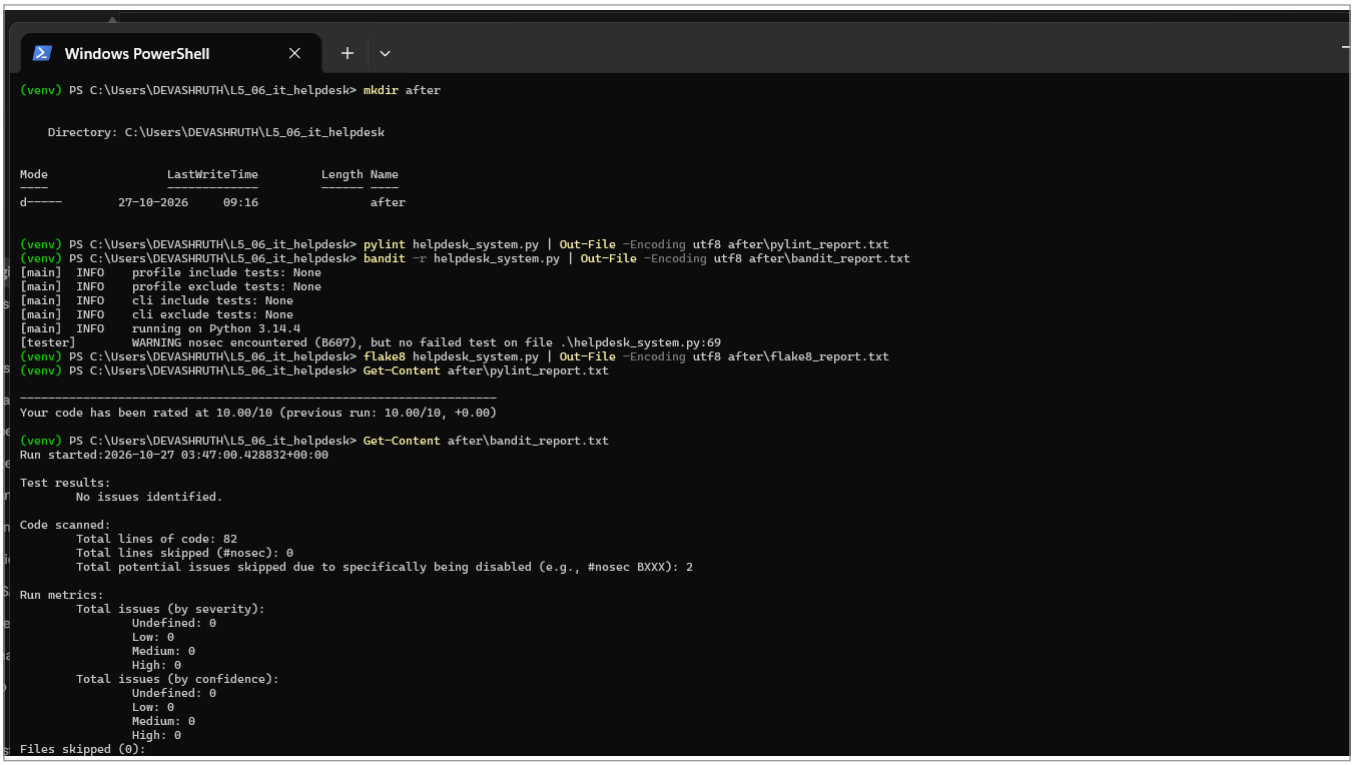


Fig 12: Pylint after: 10.00/10. Bandit after: No issues identified.

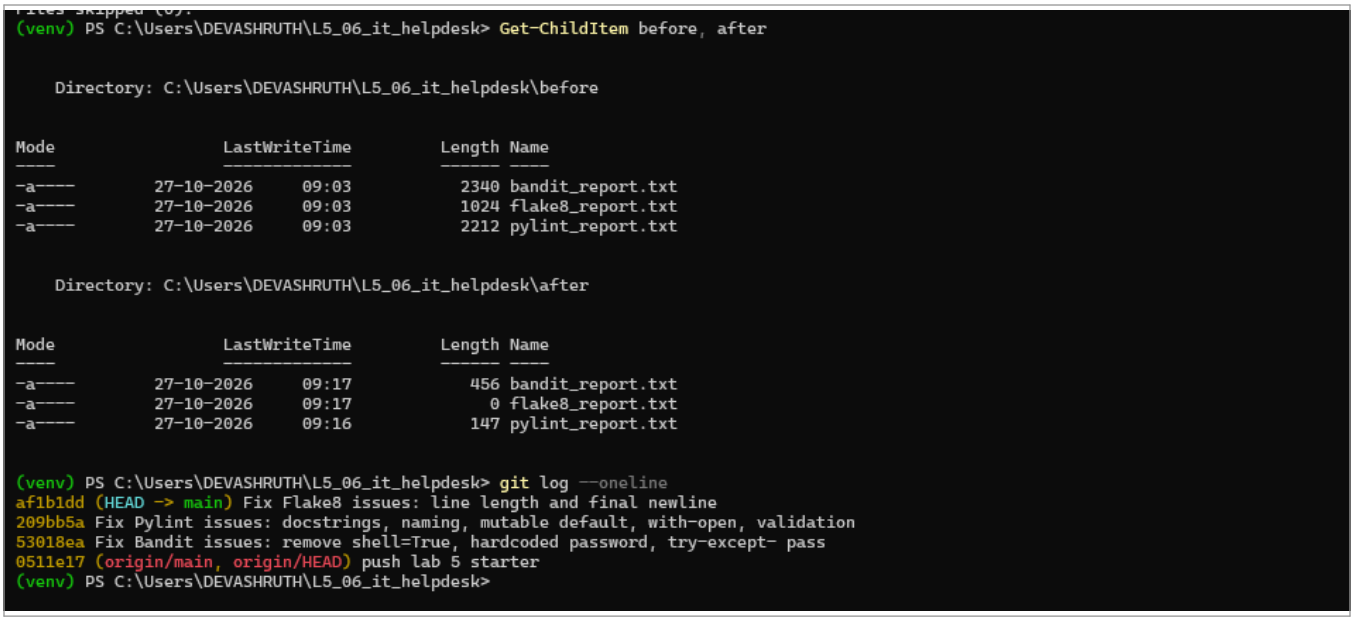


Fig 13: Before and after report files, and git log showing the three fix commits.

6. Reflection

1. What did each tool catch, and which was most useful?

Pylint found the most (21 messages) and took the score from 5.53 to 10.00. It mostly caught code quality problems like the mutable default, the bare except and missing docstrings. Flake8 only cared about formatting, mainly blank lines and long lines. Bandit gave just four findings, but the one High finding, shell=True in the ping function, was the most serious problem in the whole file. Pylint was the most useful overall, but Bandit mattered most per finding.

2. Which issues were hardest or most surprising to fix?

shell=True took the most thought. Deleting it breaks the command, so the call became an argument list, the hostname needed validating, and the ping flag differs between Windows (-n) and Linux (-c). The hardcoded password was similar: it moved to an environment variable, so the program now needs it set to close a ticket. I was also surprised that a global counter could be replaced neatly with itertools.count.

3. Did the tools give false positives, or miss anything?

A few Bandit Low findings were noise. B404 only says subprocess is imported, which is fine once it is used safely, so I marked it # nosec with a reason instead of removing a needed import. More importantly, the tools missed real bugs: a priority of -7 was accepted, a wrong password was silently ignored, and ping -c fails on Windows. I only found those by running the program and reading the output.

4. How would you use static analysis in a real project?

I would run all three tools before every commit with a pre-commit hook, and again in CI on every pull request, failing the build on any High Bandit finding or a Pylint score below a set threshold. Fix in order of severity: security first, then bugs, then style. And only suppress a warning with a written reason, never just to make the report clean.

5. What did you learn about code quality and maintainability?

The original code ran fine yet scored 5.53, so working code is not the same as good code. After the fixes it is easier to read (names, docstrings), safer, and fails loudly instead of silently. Static analysis works well as a first reviewer, but it does not replace testing or reading the program's output.

Appendix: final helpdesk_system.py

Run with: $env:HELPDESK_ADMIN_PASSWORD="helpdesk@123" then python helpdesk_system.py

```
"""IT helpdesk ticket allocator: raise, assign, note, close, save tickets."""
import itertools
import json
import os
import re
import subprocess  # nosec B404
import sys
from datetime import datetime

tickets = {}
TICKET_IDS = itertools.count(101)
ADMIN_PASSWORD = os.environ.get("HELPDESK_ADMIN_PASSWORD")


def raise_ticket(user, issue_type, urgency="3", audit_trail=None):
    """Create a ticket with urgency 1-5 and return its id."""
    priority = int(urgency)
    if not 1 <= priority <= 5:
        raise ValueError(f"Urgency must be between 1 and 5, got {priority}")
    if audit_trail is None:
        audit_trail = []
    ticket_id = next(TICKET_IDS)
    tickets[ticket_id] = {
        "user": user, "issue": issue_type, "priority": priority,
        "tech": None, "notes": [],
    }
    audit_trail.append(f"{user} raised #{ticket_id} at {datetime.now()}")
    return ticket_id


def assign_ticket(ticket_id, technician):
    """Assign a technician to a ticket."""
    tickets[ticket_id]["tech"] = technician


def add_note(ticket_id, note):
    """Add a note to a ticket, reporting if the ticket does not exist."""
    try:
        tickets[ticket_id]["notes"].append(note)
    except KeyError:
        print("Cannot add note: ticket", ticket_id, "does not exist")


def close_ticket(ticket_id, password):
    """Close (delete) a ticket if the admin password is correct."""
    if ADMIN_PASSWORD and password == ADMIN_PASSWORD:
        del tickets[ticket_id]
    else:
        print("Cannot close ticket", ticket_id, "- wrong password or not set")


def unresolved_high_priority():
    """Return ids of tickets with priority 4 or more and no technician."""
    result = []
    for ticket_id, ticket in tickets.items():
        if ticket["priority"] >= 4 and ticket["tech"] is None:
            result.append(ticket_id)
    return result


def ping_user_machine(hostname):
    """Ping a host once, after checking the hostname is safe."""
    valid = re.fullmatch(r"[A-Za-z0-9.-]+", hostname)
    if not valid or hostname.startswith("-"):
        raise ValueError("Invalid hostname")
    count_flag = "-n" if sys.platform == "win32" else "-c"
    # Safe: no shell, fixed argument list, validated hostname.
    command = ["ping", count_flag, "1", hostname]
    return subprocess.call(command)  # nosec B603 B607


def save_tickets(file="tickets.json"):
    """Save all tickets to a JSON file."""
    with open(file, "w", encoding="utf-8") as handle:
        handle.write(json.dumps(tickets))


def main():
    """Run the helpdesk demo."""
    t1 = raise_ticket("anil", "printer", "5")
    raise_ticket("meera", "wifi", "4")
    try:
        raise_ticket("joe", "vpn", "-7")
    except ValueError as error:
        print("Ticket rejected:", error)
    t3 = raise_ticket("joe", "vpn", "2")
    assign_ticket(t1, "Tech-Kiran")
    add_note(999, "Called user")
    print("Unassigned high priority:", unresolved_high_priority())
    prompt = "Urgency for a new laptop ticket (1-5, Enter for 3): "
    level = input(prompt) or "3"
    try:
        raise_ticket("sana", "laptop", level)
    except ValueError as error:
        print("Ticket rejected:", error)
    close_ticket(t3, os.environ.get("HELPDESK_ADMIN_PASSWORD", ""))
    save_tickets()
    print("Open tickets:", list(tickets.keys()))


if __name__ == "__main__":
```

```
main()
```